# Compression Ratio Is Not a Serving Metric: A Measurement Study of KV-Cache Compression for Long-Context LLM Serving

Anonymous Author(s)

## Abstract

Long-context LLM serving is bounded by the key–value (KV) cache, and a large literature compresses it. Almost all of that literature summarises a method to a prospective adopter with two numbers: a compression ratio and a task score. A serving system consumes neither; it consumes prefill latency, decode throughput and resident device memory. We ask whether the two reported numbers predict the three consumed ones, and measure that in our study they do not.

Across 3,150 paired query executions — each compressed run matched to an uncompressed run of the same model on the identical prompt — spanning seven compression methods, three open-weight models, five long-context tasks and contexts from 2K to 128K tokens, we find: (i) at an *identical* nominal budget of 4 bits per KV element, with realised payloads within 0.19 bits of one another, task accuracy ranges from 2.5% to 47.3% on one model, an 18.7× spread; (ii) two methods matched to within 0.55 effective bits per element and 2.0 accuracy points, on the same model and the same prompts, differ by 30.3× in time-to-first-token and 3.46× in peak VRAM; and (iii) over 36 operating points we detect no rank relationship between the memory a representation saves and the hardware efficiency it realises ($\rho = +0.04$, $p = 0.84$), nor between its effective footprint and its prefill cost ($\rho = +0.01$, $p = 0.98$). The reported footprint is itself systematically understated: charging a representation for the metadata that makes it decodable adds 6–23% depending on method family, and reorders the resulting ranking.

We describe TriAxis, the measurement methodology these findings required: query-level pairing against a dense reference, an effective-bits accounting that charges for mandatory metadata, and a report that keeps representation utility and measured serving efficiency on separate axes instead of merging them. We give per-method provenance, state for every method whether its saving was physically realised or analytically accounted, and scope each conclusion to the implementations and hardware measured. The per-query records and the scripts that regenerate every number and figure are released.

## 1 Introduction

A long-context LLM serving system is, to a first approximation, a memory manager. Standard attention caches a key and a value tensor for every past token at every layer and KV head, so cache memory grows linearly in sequence length $L$ and batch size $B$:

$$M_{\mathrm{KV}} = 2\, N_{\mathrm{layers}}\, N_{\mathrm{kv}}\, d_{\mathrm{head}}\, L\, B \cdot \mathrm{sizeof(dtype)}\ \text{[bytes]}. \quad (1)$$

For a Llama-3.1-8B-shaped model (32 layers, 8 KV heads, head dimension 128, 16-bit) one 128K-token sequence needs 16.0 GiB of cache alone — more than the weights. Footprint therefore sets the batch size a server sustains and the context it admits, and with them throughput and cost per token. Hence paged allocation [15], continuous batching [22], prefill/decode disaggregation [25], chunked prefill [1], compressed transfer [17] and hardware-aware compression [9]: the cache is the binding resource.

A parallel literature attacks the same resource from the representation side — quantisation [12, 19], eviction [16, 21, 24], pooling [10], low-rank projection [20], factorised state [8]. When such a mechanism is offered to a serving system it is almost always summarised by two numbers: a compression ratio (or nominal bit-width) and a task score. The implicit promise is that the first says what it costs the system and the second what it costs the user.

A serving system consumes neither number. It consumes time-to-first-token (TTFT), decode throughput and resident device memory, under a latency objective. **This paper asks whether the two reported numbers are sufficient to predict those three, and reports measurements in which they are not — by margins large enough to invert a deployment decision.**

***Finding 1: at a fixed budget, the bitrate does not order methods on task utility.*** Figure 1(a) holds five methods at the same nominal 4-bit budget: their realised payloads agree to within 0.19 bits per element and their effective footprints to within 0.55, so any accuracy difference is attributable to *how* the budget is spent, not how much. Accuracy nonetheless spans 2.5% to 47.3% on Gemma 4 E2B (18.7×), 5.3% to 61.3% on Gemma 4 E4B (11.6×) and 16.2% to 60.4% on Qwen2.5 7B (3.7×). Pooled over all 36 compressed configurations, the rank correlation between memory saved and accuracy retained is $\rho = -0.42$ ($p = 0.010$): saving more memory is, in these measurements, if anything associated with retaining *less* task ability.

***Finding 2: matched on both reported numbers, serving cost still differs by more than an order of magnitude.*** Figure 1(b) takes two methods that agree on both reported quantities — within 0.55 bits per element and 2.0 accuracy points, on the same model and the same prompts —

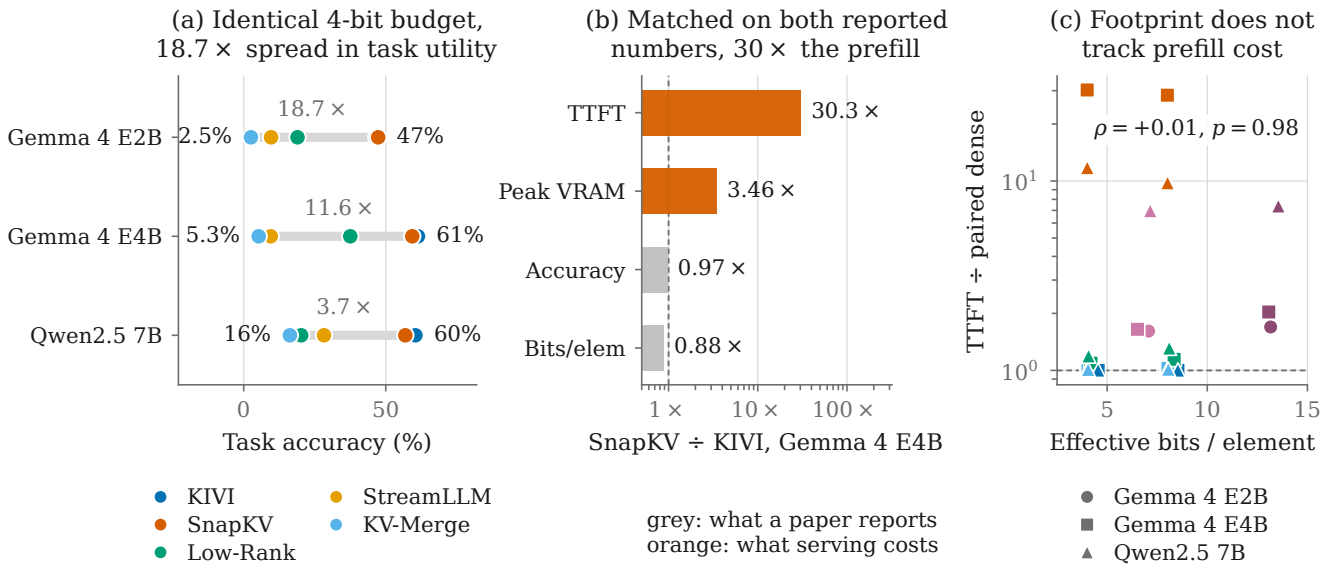



**Figure 1. The two numbers a compression paper reports do not determine what a serving system pays. (a)** Five methods held at an identical nominal budget of 4 bits per KV element — realised payloads within 0.19 bits/elem of one another, effective footprints within 0.55 once metadata is charged — differ in task accuracy by up to 18.7×. Selecting on bitrate alone leaves this entire range undetermined. **(b)** KIVI and SnapKV on Gemma 4 E4B agree on *both* reported numbers, to within 0.55 bits per element and 2.0 accuracy points, and then diverge by 3.46× in peak VRAM and 30.3× in TTFT. Grey bars are what a paper reports; orange bars are what the serving system pays. **(c)** Over all 36 compressed operating points, effective footprint carries no detectable signal about prefill cost. Marker shape denotes model, colour denotes method. Panel (b) reflects the SnapKV integration documented in §4.1, not SnapKV's fused reference kernel; the point is that a serving system inherits the integration it can run, not the algorithm in the abstract.

and finds them 3.46× apart in peak VRAM and 30.3× apart in TTFT. This is a direct matched observation, not a correlation over a scatter. Across configurations we detect no rank relationship between memory saved and the measured hardware efficiency multiplier ($\rho = +0.04$, $p = 0.84$) or between effective footprint and prefill cost ($\rho = +0.01$, $p = 0.98$). We read these as the absence of a *detectable* relationship at this sample size, not as proof that none exists; the matched comparison does not depend on them.

***Finding 3: the reported footprint is itself understated, unevenly.*** A representation's real cost is its payload *plus* the metadata that makes the payload decodable: quantisation scales and zero points, subspace bases, eviction index tables, alignment padding. Charging for it moves a factorised representation from a nominal 5.00 to a realised 6.51 bits per element and a nominal 4-bit quantiser from 4.05 to 4.55 — understatements of 30% and 12%, both flattering the method. Because the overhead concentrates in particular families (0.1% for eviction, 6–11% for grouped quantisation, 10–23% for factorised state) rather than spreading evenly, correcting it does not merely rescale the ranking, it reorders it.

***Why this matters for serving systems.*** Each finding breaks a decision a serving system actually makes. *Capacity planning* is sized from resident memory, and we observe peak resident memory diverging from the analytic footprint by up to 3.46× on the same model, so a plan drawn from a compression ratio can err in the unsafe direction. *Latency objectives* are set on TTFT, and prefill-side transformation work — attention-score pooling, SVD, block factorisation — runs over the *uncompressed* prompt before any saving materialises, which no bitrate expresses. *Mechanism selection* is per-deployment: the same integration inflates TTFT 30.3× on Gemma 4 E4B, 11.8× on Qwen2.5 7B and 1.04× on Gemma 4 E2B, where its cost surfaces instead as a 2.47× peak-VRAM inflation — so a single-model measurement would have supported a confidently wrong generalisation in either direction. In short, what a serving system runs is an *implementation*, and its cost is not recoverable from the algorithm's published ratio.

***Contributions.*** None of this is visible under existing practice, because task quality, memory and latency are conventionally measured by different communities on different workloads, so their disagreements never appear on the same

row. Establishing that they disagree requires measuring all three *on the same query*, and that measurement design — not the summary scores it produces — is what makes the findings above credible.

- **An empirical characterisation of a three-way divergence that the field reports as two numbers** (§5): at matched bitrate, accuracy varies up to 18.7×; at matched bitrate *and* accuracy, serving cost varies 30.3× in TTFT and 3.46× in peak VRAM; over 36 operating points we detect no rank association between representation footprint and realised hardware behaviour; and the divergence is itself model-dependent.
- **TriAxis, the measurement methodology that makes that characterisation possible** (§3): a paired-execution primitive that runs every query twice, once under the 16-bit dense reference and once under the candidate, on the identical prompt in the same session, so that every ratio is formed against a baseline that cannot be differently configured or differently reported.
- **An honest memory accounting** (§3.3): effective bits per element, charging each representation for the metadata without which its payload cannot be decoded, plus an explicit separation of *analytically accounted* from *physically realised* savings. This is load-bearing rather than pedantic: it shifts reported bitrates by 6–23% and changes rankings.
- **Per-method provenance and a reproducible pipeline** (§4.1, Appendix E): for every method we state which code path produced its numbers and what that path does and does not represent, and we release the per-query records with scripts that regenerate every table, statistic and figure here.

***What we do not claim.*** This is a controlled single-node study: three open-weight models (2B–7B), one commodity GPU, batch size one, no continuous batching or paged attention. We show that the reported-number-to-serving-cost mapping breaks in this regime and quantify by how much; we do not claim the magnitudes transfer to other model scales or to multi-tenant stacks, and §7 is explicit about the boundary. Concurrent work independently reports that compression ratio is a poor predictor of end-to-end performance on a different model and task set [2]; §8 states what our measurement design adds to that observation.

## 2 Three Axes, and Why One Does Not Determine Another

Current practice reports two numbers where a deployment decision needs three. *Representation efficiency* (Axis 1) is how many bits of state a method keeps per KV element, *including* everything needed to decode that state; it is a property of the representation alone, it is what a compression ratio tries to express, and it determines how much more context fits in a fixed memory budget. *Task utility* (Axis 2) is how much of the model's ability survives the transform; it is a property of the representation *and* the model *and* the task. *Serving efficiency* (Axis 3) is what the method costs in prefill latency, decode throughput and resident device memory when actually executed; it is a property of the representation, its *implementation*, and the hardware. Only the first can be read off a datasheet.

Axis 1 need not determine Axis 2 because two representations can keep the same number of bits and spend them completely differently: a 4-bit quantiser spends its budget uniformly over all tokens, an eviction policy spends 16 bits on a quarter of the tokens and nothing on the rest, and a pooling scheme spends them on averages that may correspond to no token the model needs. Which allocation is right depends on where the task's information lies, which a bit count cannot express.

Axis 1 need not determine Axis 3 for four reasons, each of which we measure. *(i) Metadata is neither free nor proportional.* Group-wise quantisation stores a scale and zero point per group; a factorised scheme stores anchors and bases; an eviction method needs only a position index. At a nominal 4-bit budget these cost 11%, 23% and 0.1% of realised footprint respectively (§5.2), so a ratio computed on payload alone mis-ranks them. *(ii) Prefill cost is decided by the transform, not the footprint.* Attention-score pooling, SVD and block factorisation all do work proportional to the *uncompressed* prompt before any saving materialises; our measured TTFT inflation spans 0.96× to 30.3× dense at effectively identical footprints. *(iii) Implementation efficiency is part of the answer.* The same algorithm as a fused kernel or as un-fused tensor operations is a different serving proposition — and it is the deployable artifact, not the algorithm, that a serving system runs. *(iv) Peak memory is not resident memory.* A transform that ends smaller can pass through a much larger intermediate; in our runs the method with the smallest analytic footprint at 4 bits had the largest measured peak VRAM.

The three axes therefore have to be measured separately, on the same queries. That is what TriAxis does.

## 3 TriAxis: Measuring the Three Axes on One Query

### 3.1 The paired execution primitive

Let $\mathcal{M}$ be an autoregressive model with fixed weights, $\mathcal{D} = \{(x_i, y_i, \tau_i, L_i)\}_{i=1}^{N}$ a set of queries with targets, task types and context lengths, $\mathcal{R}_{\text{dense}}$ the uncompressed 16-bit reference representation, and $\mathcal{A}(\mathcal{B})$ a candidate representation under budget $\mathcal{B}$. The atomic unit of measurement is the 4-tuple $\mathcal{U}_i = (\mathcal{M}, (x_i, y_i), \mathcal{R}_{\text{dense}}, \mathcal{A}(\mathcal{B}))$. For each unit TriAxis executes both $\hat{y}_{i,\text{dense}} = \mathcal{M}(x_i \mid \mathcal{R}_{\text{dense}})$ and $\hat{y}_{i,\text{method}} = \mathcal{M}(x_i \mid \mathcal{A}(\mathcal{B}))$ on the identical prompt, and records for both the task score $s \in [0, 1]$, the analytic state footprint in bytes,

and the measured TTFT, decode throughput and peak device memory (Figure 2).

Pairing at the query level rather than the dataset level is what makes the three axes comparable. Every ratio we report — accuracy, TTFT inflation, VRAM inflation — is formed against a dense execution of the same prompt in the same session on the same device, so it cannot be contaminated by a differently configured or differently reported baseline. It also means a per-query anomaly is attributable: the dense and compressed outcomes for that exact prompt are both on record.

### 3.2 Task quality

For a completed candidate run with evaluator score $s_{i,\text{method}} \in [0,1]$ we report absolute success, $Q_{\text{abs},i} = 100\, s_{i,\text{method}}$ clipped to $[0,100]$, rather than retention relative to dense. Retention rewards a method for being paired with a weak baseline: a method scoring 20% where dense scores 20% has 100% retention and is useless. The paired dense outcome is still recorded for every query — it defines coverage, enables failure diagnosis, and lets a reader recompute retention — but it is not substituted into the headline number. We log the signed paired difference $\Delta_i = s_{i,\text{method}} - s_{i,\text{dense}}$ throughout.

### 3.3 Memory accounting: effective bits per element

This is the axis most often misreported, so we state the accounting explicitly. The total state of a representation is

$$M_{\text{method}}(L) = \underbrace{M_{\text{algo}}(L)}_{\text{payload}} + \underbrace{M_{\text{meta}}(L)}_{\text{scales, bases, indices}} + \underbrace{M_{\text{align}}(L)}_{\text{padding}}, \quad (2)$$

and normalising the *total* by the uncompressed element count gives

$$b_{\text{eff}} = \frac{8 \cdot M_{\text{method}}(L)}{2\, N_{\text{layers}}\, N_{\text{kv}}\, d_{\text{head}}\, L} \quad \text{[bits/element]}, \quad (3)$$

which is exactly 16.0 for an uncompressed FP16/BF16 cache. Memory saving is $R_{\text{mem}} = 100 \cdot \max(0,\, 1 - M_{\text{method}}/M_{\text{dense}})$.

A method's nominal bit-width describes only $M_{\text{algo}}$. The remaining terms are not optional: without the scales the payload cannot be dequantised, without the bases the projection cannot be inverted. They are also not uniform across families — group-wise quantisation pays per group, a subspace method per block, an eviction method only a token index — so omitting them does not shift every method equally, it *re-ranks* them (§5.2).

***Analytic versus physically realised savings.*** We distinguish two things a compression result can mean. A saving is *physically realised* when the resident cache tensors actually shrink, and *analytically accounted* when the scheme's footprint is computed from its definition while the harness retains the transformed values in a wider dtype for exactness. Both are legitimate — the first measures a system, the second measures a representation — but they are not interchangeable, and conflating them is how a paper ends up claiming a serving win it did not measure. §4.1 states which is which for every method here, and every peak-VRAM number we report reflects what was physically resident.

### 3.4 Two tracks, and which one carries the evidence

***Part 1 (representation and task).***

$$\mathcal{S}_{\text{res}} = \alpha\, Q_{\text{abs}} + (1-\alpha)\, R_{\text{mem}}, \qquad \alpha = 0.70 \text{ by default.} \quad (4)$$

The linear form is chosen for transparency, not optimality: it is monotone in both arguments, bounded on $[0,100]$, Pareto consistent, and — unlike a product or harmonic form — never lets one dimension annihilate the other, so a reader can always decompose a score back into the two quantities that produced it (Appendices A and B). At $\alpha = 0.70$ one accuracy point is worth 2.33 points of memory saving; we report $\alpha$ sensitivity in Appendix B rather than asking the reader to accept the default. Aggregation uses a byte-weighted memory aggregate so that long contexts are not diluted by short ones, and failed queries are charged the dense footprint and score zero rather than being dropped (Appendix D). Paired coverage $C = N_{\text{paired success}}/N_{\text{total}}$ is reported *beside* $\mathcal{S}_{\text{res}}$ rather than multiplied into it, so a run that failed for allocator reasons stays distinguishable from one that completed and scored zero. Coverage was 1.00 for every configuration reported here.

***Part 2 (measured serving).*** Part 2 records TTFT (wall-clock to first emitted token, including all transform work), decode throughput $\mathcal{T}_{\text{dec}}$, and peak resident device memory $\mathcal{P}$. Each is converted to a paired efficiency against the *same query's* dense execution,

$$\begin{aligned} \phi_{\text{ttft}} &= \min\big(1,\ \text{TTFT}_{\text{dense}}/\text{TTFT}_{\text{method}}\big), \\ \phi_{\text{thru}} &= \min\big(1,\ \mathcal{T}_{\text{method}}/\mathcal{T}_{\text{dense}}\big), \\ \phi_{\text{vram}} &= \min\big(1,\ \mathcal{P}_{\text{dense}}/\mathcal{P}_{\text{method}}\big), \end{aligned} \quad (5)$$

and combined into a single *hardware multiplier*

$$\phi = \phi_{\text{ttft}}^{0.35}\, \phi_{\text{thru}}^{0.35}\, \phi_{\text{vram}}^{0.30} \in (0,1]. \quad (6)$$

The weights give interactive responsiveness slightly more emphasis than resident footprint; we make no claim this is the only defensible weighting, and the components are always reported separately so a reader can reweight. $\phi$ is computed per query and then averaged.

***$\phi$ is the serving axis; $\mathcal{S}_{\text{sys}}$ is only a convenience. $\phi$ contains no term from Part 1.*** It is built entirely from paired latency, throughput and memory measurements, and it is therefore the quantity to use whenever the question is whether algorithmic utility predicts serving behaviour. For readers who want a single ordering we also report

$$\mathcal{S}_{\text{sys}} = \alpha \mathcal{S}_{\text{res}} + (1-\alpha) \cdot 100\phi, \quad (7)$$

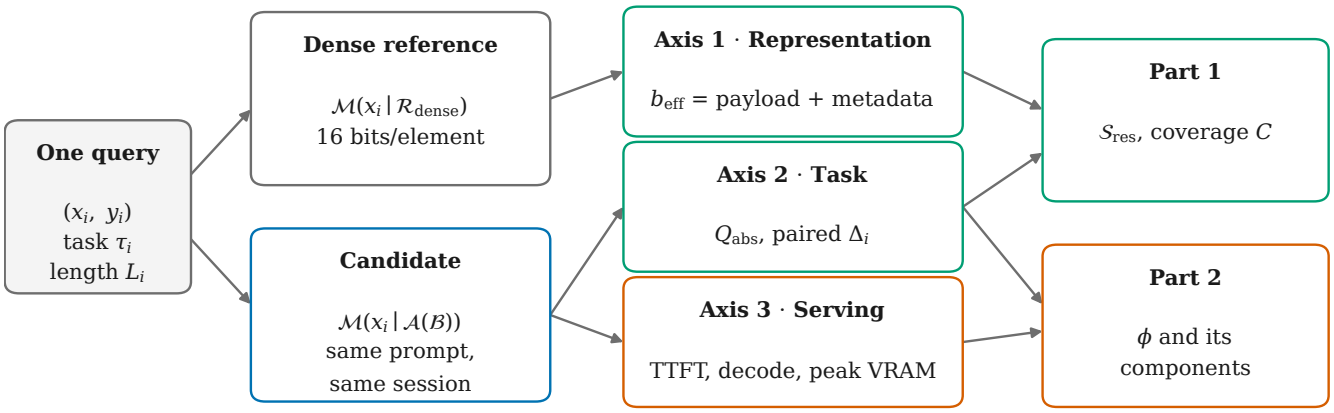



**Figure 2. Every measurement in this paper is a difference between two executions of the same prompt.** The dense reference is re-run for each query rather than quoted from a previous session, which is what allows an accuracy difference, a footprint difference and a latency difference to be attributed to the representation rather than to the harness. Part 1 and Part 2 are reported separately by design (§3.4).

but $\mathcal{S}_{\text{sys}}$ is a presentational summary, *not* evidence: it is defined in terms of $\mathcal{S}_{\text{res}}$, so any correlation between the two is circular by construction and we never compute one. Every independence claim in this paper is made against $\phi$. One consequence is worth flagging in advance: the rank displacements we report in §5.3 between the $\mathcal{S}_{\text{res}}$ ordering and the $\mathcal{S}_{\text{sys}}$ ordering are a *lower bound* on the disagreement between the axes, precisely because the two orderings share the $\mathcal{S}_{\text{res}}$ term. Nothing in the paper's argument would change if $\mathcal{S}_{\text{sys}}$ were deleted; it is retained only because a single column is convenient to scan.

## 4 Study Design

***Design.*** Each model is evaluated on 5 tasks × 5 context lengths × 3 samples = 75 distinct queries. Each query is executed under 14 configurations: five fixed-budget methods at nominal budgets of 4 and 8 bits per element, two adaptive factorised configurations, and the dense reference (recorded in both budget sweeps). That is 1,050 paired executions per model and 3,150 in total. Decoding is greedy ($T = 0$) and the generation budget is fixed per task (32–256 new tokens) and identical across every configuration, so no method is advantaged by a longer or shorter budget. We report operating points separately rather than pooling the two budget sweeps into one row per method: a pooled "≈ 6 bits" row averages two configurations whose accuracies can differ by more than 30 points, and it is precisely the matched-budget comparison that carries the evidence.

***Models and context ladders.*** Gemma 4 E2B (`google/gemma-4-e2b-it`) at 2K, 8K, 32K, 64K, 128K; Gemma 4 E4B (`google/gemma-4-e4b-it`) and Qwen2.5 7B (`Qwen/Qwen2.5-7B-Instruct`) at 2K, 4K, 8K, 16K, 32K.

***Tasks.*** Single-needle and multi-needle retrieval at parameterised depths [14]; RULER key–value lookup among distractors and RULER multi-step variable tracking [13]; and synthetic cross-document multi-hop QA. Two of the five behave as controls rather than discriminators in our study, and §5.4 treats them as such.

***Environment.*** A single NVIDIA GeForce RTX 4070 SUPER (12 GB VRAM), PyTorch 2.6.0 (CUDA 12.4 build), `transformers` 5.15.1, Python 3.12.10, Windows 11. Weights were loaded in 4-bit NormalFloat via bitsandbytes so that the weight footprint is held constant and the 128K ladder fits in 12 GB; the KV cache under study is unaffected by that choice. Prefill used chunked forward passes (4,096 tokens) to avoid intermediate allocation spikes. Absolute latencies are hardware-specific, but because every comparison is paired against a dense execution on the same device in the same session, the reported ratios are not distorted by machine-to-machine variation.

***Determinism.*** The dense reference was executed twice for every query, once in each budget sweep, which gives a free replication check: on Gemma 4 E4B and Qwen2.5 7B all 150 re-executions produced byte-identical outputs, and on Gemma 4 E2B exactly one query of 75 changed score. Run-to-run variance is negligible at the resolution of our claims (Appendix D).

### 4.1 Methods and provenance

Table 1 states, for every method, which implementation produced its numbers and whether its memory saving was physically realised or analytically accounted. We treat this

disclosure as part of the result rather than an appendix detail: a serving claim is only as strong as the code path that produced it.

Two caveats deserve to be read before any serving number in this paper. Driven outside its fused attention path, SnapKV's observation-window pooling runs as separate tensor operations over the full prompt, which is why its measured prefill cost is large. We report it because it is the honest measurement of the code path that was runnable here, and because it is a direct instance of the paper's own thesis: serving efficiency is a property of an algorithm *and* its implementation, and is therefore not predictable from a compression ratio. It must not be read as a claim about SnapKV's achievable latency. The same caveat applies in the opposite direction to DKV: its representation is measured, its serving runtime is not.

# 5 Results

The three subsections below take the three questions of §1 in turn. Table 2 is the study in one grid — all three axes at every operating point, for all three models — and every comparison drawn below can be read off it directly.

## 5.1 Does the bitrate order methods on task utility?

Holding the nominal budget fixed isolates Axis 1 from Axis 2: at 4 bits the five fixed-budget methods realise payloads within 0.19 bits per element of one another, and effective footprints within 0.55 once metadata is charged, so any accuracy difference between them is attributable to how the budget is spent rather than how much.

The differences are large. On Gemma 4 E2B accuracy at 4 bits ranges from 2.5% (KV-Merging) to 47.3% (SnapKV), a factor of 18.7; on Gemma 4 E4B from 5.3% to 61.3% (11.6×); on Qwen2.5 7B from 16.2% to 60.4% (3.7×). At 8 bits the spreads narrow but remain substantial (22.2, 21.2 and 37.4 accuracy points respectively), because the methods that fail at 4 bits recover unevenly: StreamingLLM gains 26.5 points on E2B between the two budgets while KIVI gains none (46.7% → 46.2%, within noise). How much accuracy a bit buys is itself method-dependent.

Table 3 shows the same comparison across models, and the ordering is not stable: Low-Rank KV is third at 4 bits on the two Gemma models and fourth on Qwen2.5 7B, where StreamingLLM passes it; and at 8 bits StreamingLLM is last of the five on Gemma 4 E4B (41.4%) yet third on Qwen2.5 7B (44.8%). Pooled over all 36 compressed operating points the rank correlation between memory saved and accuracy is $\rho = -0.42$ ($p = 0.010$); per model it is −0.28, −0.64 and −0.39. The negative sign is unsurprising — within a family, saving more costs accuracy — but what matters here is the absence of any positive predictive relationship *across* families. In these measurements, a bitrate is not sufficient to order these methods on task ability, even approximately.

## 5.2 How much does the metadata cost?

Figure 3 decomposes every operating point into payload and mandatory metadata. The overhead is strongly method-dependent. Eviction and pooling methods store only a token index and pay 0.1–0.4%. Grouped quantisation pays for scales and zero points: a nominal 4-bit configuration realises 4.51–4.55 bits per element (11% overhead), and because the metadata is per group rather than per bit, the same scheme at 8 bits pays only 5.8–5.9%. The factorised representations pay most — 18–23% on the two Gemma models and 10–18% on Qwen2.5 7B — moving DKV-Mid from a nominal 5.00 to a realised 6.51 bits per element on Gemma 4 E4B and DKV-High from 10.68 to 13.07 — at which point it saves 18% of the dense footprint rather than the 33% a payload-only accounting would report.

This matters for ranking, not only for absolute numbers. Because the overhead concentrates in one family, charging for it reorders the table: on Gemma 4 E4B, DKV-Mid falls from third to fifth of thirteen operating points, passed by both 8-bit configurations it had outranked; on Gemma 4 E2B and Qwen2.5 7B, DKV-High falls one position, passed by 8-bit StreamingLLM. The eviction and pooling methods are barely charged at all, having almost nothing to charge, so where one of them moves it is because a quantised or factorised neighbour fell past it rather than because its own footprint changed. We note that the correction is largest for the family whose numbers it most penalises, and we report it in that direction because that is what the accounting yields.

## 5.3 Does task utility predict serving efficiency?

The clearest evidence is a matched pair rather than a correlation. On Gemma 4 E4B at a 4-bit budget, KIVI and SnapKV agree closely on both conventionally reported numbers — 4.55 versus 4.00 bits per element, 61.3% versus 59.3% accuracy — and then diverge completely on the third axis: TTFT of 2.84 s versus 86.04 s (30.3×), peak VRAM of 761 MB versus 2,632 MB (3.46×), and hardware multiplier $\phi$ of 0.99 versus 0.48. The same pair on Qwen2.5 7B differs by 11.7× in TTFT. Nothing in Axis 1 or Axis 2 anticipates a 30× prefill gap.

Across all operating points the association is not merely weak; it is not detectable at all at this sample size. Memory saved has no rank relationship to the measured hardware multiplier ($\rho = +0.04$, $p = 0.84$); effective footprint has none to TTFT inflation ($\rho = +0.01$, $p = 0.98$) or to measured peak-VRAM inflation ($\rho = -0.08$, $p = 0.63$). Within each model, algorithmic utility is if anything mildly *anti*-correlated with hardware efficiency ($\rho(S_{\text{res}}, \phi) = -0.33, -0.30, -0.49$), which is what one would expect if the methods that work hardest to preserve quality also do the most work at prefill. With 12–13 points per model these are descriptive summaries of a small sample; we quote them to indicate the

**Table 1. A serving number is a property of a code path, so we say which one.** "Upstream" means the authors' released code was invoked; "harness" means a reference implementation written for this study following the published scheme. "Realised" means the resident cache tensors physically shrank; "analytic" means the footprint was computed from the scheme while values were retained in a wider dtype, so quality is exact for the scheme but no device memory was freed.

| Method | Impl. | Saving | Budget | What the measured numbers do and do not represent |
|---|---|---|---|---|
| KIVI (INT4/INT8) | harness | analytic | 4, 8 | Asymmetric grouped min–max, per-channel keys, per-token values [19]; partial key groups stay at 16 bits and are charged as such. Quality is exact for the scheme; its favourable $\phi$ reflects a cheap transform, not a production integer kernel. |
| SnapKV | upstream | realised | 4, 8 | `SnapKVCluster.update_kv` at `e216ddc`. Upstream's monkeypatch targets `transformers` 4.37 and Llama/Mistral only, so under `transformers` 5.x the cluster is driven directly, *outside* the fused attention path. Its TTFT is a property of that integration, not of SnapKV's achievable latency. |
| StreamingLLM | upstream | realised | 4, 8 | `StartRecentKVCache` at `2e50426`; policy unmodified. |
| KV-Merging | harness | realised | 4, 8 | Temporal mean-pooling of consecutive states, sinks and recent window preserved [10]. |
| Low-Rank KV | harness | analytic | 4, 8 | Per-head mean-centred truncated SVD [20]; footprint is the $U, \Sigma, V$ factor size. |
| DKV-Mid / DKV-High | upstream | analytic | adaptive | Anchor-plus-low-rank differential compression [8]. The block compressor is driven over the resident cache; the repository's paged serving runtime and decode kernel were unavailable on this platform, so these numbers characterise the *representation*, not that runtime's decode speedup. |
| Dense FP16 | — | — | 16 | Uncompressed paired reference. |

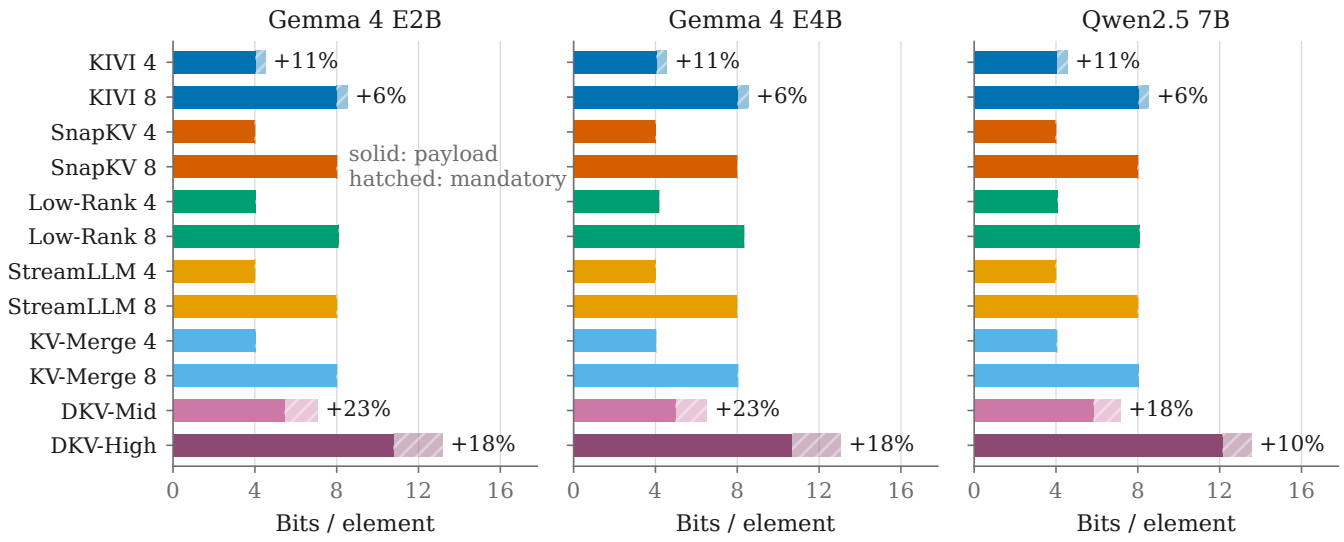



**Figure 3. The bits a method is charged for and the bits it actually needs are not the same number, and the gap is family-specific.** Solid bars are the compressed payload conventionally reported as the compression ratio; hatched caps are the metadata without which the payload cannot be decoded. Labels give the metadata share where it exceeds 5%. Because eviction carries almost none and factorised state carries 10–23%, a payload-only ratio does not merely understate footprint — it reorders methods against one another.

absence of a detectable relationship, not to assert that none exists.

Consequently the two tracks order methods differently. Moving from the $\mathcal{S}_{\text{res}}$ ordering to the $\mathcal{S}_{\text{sys}}$ ordering shifts

**Table 2. Reading across a row shows the three axes disagreeing.** $b_{\text{pay}}$ is the compressed payload alone — the quantity conventionally reported — and $b_{\text{eff}}$ additionally charges mandatory metadata (Eq. 3); "meta" is the metadata share of realised footprint. "×dense" is the mean per-query TTFT ratio against that query's own dense execution. $\phi$ (Eq. 6) is the pure hardware term and carries the serving evidence; $\mathcal{S}_{\text{sys}}$ is the convenience summary of §3.4 and is not used as evidence anywhere. Best $\mathcal{S}_{\text{res}}$ and best $\phi$ per model in **bold** ($\phi$ excluding the dense reference, which is 1.00 by definition). Serving figures reflect the implementations in Table 1. Paired coverage was 1.00 throughout.

| | | Axis 1: representation | | | Axis 2: task | | Axis 3: measured serving | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Method** | **Bits** | $b_{\text{pay}}$ | $b_{\text{eff}}$ | meta (%) | Acc. (%) | $\mathcal{S}_{\text{res}}$ | TTFT (s) | ×dense | Dec. (tok/s) | VRAM (MB) | $\phi$ | $\mathcal{S}_{\text{sys}}$ |
| *Gemma 4 E2B* (2K–128K) | | | | | | | | | | | | |
| KIVI | 4 | 4.01 | 4.51 | 11.1 | 46.7 | 54.2 | 12.51 | 1.00× | 12.1 | 846 | **0.97** | 67.1 |
| KIVI | 8 | 8.01 | 8.51 | 5.9 | 46.2 | 46.4 | 12.51 | 1.00× | 11.7 | 848 | 0.96 | 61.2 |
| SnapKV | 4 | 4.00 | 4.01 | 0.2 | 47.3 | **55.6** | 13.02 | 1.04× | 12.8 | 2088 | 0.71 | 60.2 |
| SnapKV | 8 | 8.00 | 8.02 | 0.2 | 46.2 | 47.3 | 13.06 | 1.04× | 13.0 | 2088 | 0.71 | 54.4 |
| Low-Rank | 4 | 4.05 | 4.05 | 0.0 | 18.9 | 35.7 | 12.76 | 1.02× | 12.3 | 849 | 0.92 | 52.5 |
| Low-Rank | 8 | 8.09 | 8.09 | 0.0 | 43.6 | 45.3 | 12.84 | 1.02× | 12.2 | 851 | 0.91 | 59.1 |
| StreamLLM | 4 | 4.00 | 4.01 | 0.2 | 9.6 | 29.2 | 12.58 | 1.00× | 13.6 | 847 | 0.95 | 48.8 |
| StreamLLM | 8 | 8.00 | 8.02 | 0.2 | 36.1 | 40.2 | 12.58 | 1.00× | 13.7 | 848 | 0.95 | 56.7 |
| KV-Merge | 4 | 4.01 | 4.02 | 0.4 | 2.5 | 24.2 | 12.60 | 1.00× | 14.0 | 848 | 0.95 | 45.3 |
| KV-Merge | 8 | 8.01 | 8.04 | 0.4 | 24.0 | 31.7 | 12.57 | 1.00× | 13.5 | 848 | 0.95 | 50.6 |
| DKV-Mid | adapt. | 5.46 | 7.07 | 22.8 | 44.7 | 48.0 | 22.26 | 1.61× | 13.8 | 1390 | 0.84 | 58.9 |
| DKV-High | adapt. | 10.78 | 13.17 | 18.1 | 46.7 | 38.0 | 23.40 | 1.70× | 13.8 | 1391 | 0.83 | 51.4 |
| Dense FP16 | – | 16.00 | 16.00 | 0.0 | 46.4 | 32.5 | 13.18 | 1.00× | 13.4 | 1118 | 1.00 | 52.8 |
| *Gemma 4 E4B* (2K–32K) | | | | | | | | | | | | |
| KIVI | 4 | 4.05 | 4.55 | 10.9 | 61.3 | **64.4** | 2.84 | 1.00× | 12.8 | 761 | **0.99** | 74.7 |
| KIVI | 8 | 8.03 | 8.53 | 5.8 | 62.7 | 57.9 | 2.84 | 1.00× | 12.8 | 761 | 0.99 | 70.1 |
| SnapKV | 4 | 4.00 | 4.00 | 0.1 | 59.3 | 64.0 | 86.04 | 30.26× | 11.2 | 2632 | 0.48 | 59.1 |
| SnapKV | 8 | 8.00 | 8.01 | 0.1 | 60.7 | 57.5 | 80.73 | 28.39× | 10.0 | 2632 | 0.47 | 54.4 |
| Low-Rank | 4 | 4.19 | 4.19 | 0.0 | 37.5 | 48.4 | 3.11 | 1.09× | 13.1 | 761 | 0.93 | 61.9 |
| Low-Rank | 8 | 8.34 | 8.34 | 0.0 | 60.0 | 56.4 | 3.25 | 1.14× | 13.0 | 761 | 0.91 | 66.7 |
| StreamLLM | 4 | 4.00 | 4.00 | 0.1 | 9.5 | 29.1 | 2.92 | 1.03× | 13.6 | 761 | 0.97 | 49.4 |
| StreamLLM | 8 | 8.00 | 8.01 | 0.1 | 41.4 | 44.0 | 2.91 | 1.02× | 13.6 | 762 | 0.97 | 59.8 |
| KV-Merge | 4 | 4.03 | 4.04 | 0.2 | 5.3 | 26.1 | 2.91 | 1.02× | 13.7 | 761 | 0.97 | 47.3 |
| KV-Merge | 8 | 8.02 | 8.04 | 0.2 | 45.3 | 46.7 | 2.92 | 1.03× | 13.3 | 762 | 0.97 | 61.6 |
| DKV-Mid | adapt. | 5.00 | 6.51 | 23.2 | 55.3 | 56.5 | 5.06 | 1.65× | 12.3 | 761 | 0.84 | 64.8 |
| DKV-High | adapt. | 10.68 | 13.07 | 18.3 | 61.3 | 48.4 | 6.25 | 2.03× | 12.2 | 761 | 0.85 | 59.3 |
| Dense FP16 | – | 16.00 | 16.00 | 0.0 | 64.0 | 44.8 | 2.96 | 1.00× | 12.4 | 761 | 1.00 | 61.4 |
| *Qwen2.5 7B* (2K–32K) | | | | | | | | | | | | |
| KIVI | 4 | 4.05 | 4.55 | 10.9 | 60.4 | **63.8** | 5.47 | 1.00× | 16.4 | 3854 | 0.98 | 74.1 |
| KIVI | 8 | 8.04 | 8.53 | 5.8 | 60.9 | 56.6 | 5.46 | 1.00× | 16.4 | 3854 | 0.98 | 69.2 |
| SnapKV | 4 | 4.00 | 4.01 | 0.2 | 56.9 | 62.3 | 64.08 | 11.77× | 24.3 | 5128 | 0.79 | 67.4 |
| SnapKV | 8 | 8.00 | 8.02 | 0.2 | 59.6 | 56.7 | 53.30 | 9.79× | 20.0 | 5137 | 0.77 | 62.6 |
| Low-Rank | 4 | 4.07 | 4.07 | 0.0 | 20.2 | 36.5 | 6.49 | 1.19× | 18.2 | 3854 | 0.91 | 52.8 |
| Low-Rank | 8 | 8.10 | 8.10 | 0.0 | 36.9 | 40.6 | 7.14 | 1.31× | 18.0 | 3854 | 0.87 | 54.4 |
| StreamLLM | 4 | 4.00 | 4.01 | 0.2 | 28.3 | 42.3 | 5.50 | 1.01× | 27.1 | 3854 | 0.99 | 59.3 |
| StreamLLM | 8 | 8.00 | 8.02 | 0.2 | 44.8 | 46.3 | 5.53 | 1.02× | 22.8 | 3854 | 0.99 | 62.1 |
| KV-Merge | 4 | 4.03 | 4.05 | 0.4 | 16.2 | 33.8 | 5.51 | 1.01× | 28.1 | 3854 | **0.99** | 53.4 |
| KV-Merge | 8 | 8.02 | 8.05 | 0.4 | 23.5 | 31.4 | 5.52 | 1.01× | 23.6 | 3854 | 0.99 | 51.7 |
| DKV-Mid | adapt. | 5.84 | 7.15 | 18.3 | 53.3 | 53.9 | 39.55 | 6.97× | 15.3 | 3854 | 0.70 | 58.8 |
| DKV-High | adapt. | 12.14 | 13.55 | 10.4 | 59.1 | 46.0 | 41.88 | 7.38× | 15.3 | 3854 | 0.73 | 54.0 |
| Dense FP16 | – | 16.00 | 16.00 | 0.0 | 60.0 | 42.0 | 5.56 | 1.00× | 16.3 | 3854 | 1.00 | 59.4 |

a configuration by 1.4, 2.6 and 1.4 positions on average for the three models; the largest single move is SnapKV at 4 bits on Gemma 4 E4B, second of thirteen on representation utility and tenth on the serving summary. As noted in §3.4,

**Table 3. Identical budget, three models, and no stable ordering.** Accuracy per model at each controlled nominal budget, with the mean realised $b_{\text{eff}}$, mean paired TTFT inflation and mean hardware multiplier over the three models. Within a block every method holds the same nominal budget, so accuracy differences are attributable to how the budget is spent, and the serving columns to how the transform is implemented.

| | Task accuracy (%) | | | | Serving | |
|---|---|---|---|---|---|---|
| **Method** | E2B | E4B | 7B | $b_{\text{eff}}$ | TTFT | $\phi$ |
| *Identical nominal budget: 4 bits/element* | | | | | | |
| KIVI | 46.7 | 61.3 | 60.4 | 4.54 | 1.00× | 0.98 |
| SnapKV | 47.3 | 59.3 | 56.9 | 4.01 | 14.36× | 0.66 |
| Low-Rank | 18.9 | 37.5 | 20.2 | 4.11 | 1.10× | 0.92 |
| StreamLLM | 9.6 | 9.5 | 28.3 | 4.01 | 1.01× | 0.97 |
| KV-Merge | 2.5 | 5.3 | 16.2 | 4.04 | 1.01× | 0.97 |
| *Identical nominal budget: 8 bits/element* | | | | | | |
| KIVI | 46.2 | 62.7 | 60.9 | 8.52 | 1.00× | 0.98 |
| SnapKV | 46.2 | 60.7 | 59.6 | 8.01 | 13.08× | 0.65 |
| Low-Rank | 43.6 | 60.0 | 36.9 | 8.18 | 1.16× | 0.90 |
| StreamLLM | 36.1 | 41.4 | 44.8 | 8.01 | 1.01× | 0.97 |
| KV-Merge | 24.0 | 45.3 | 23.5 | 8.04 | 1.01× | 0.97 |

these displacements understate the disagreement, because $\mathcal{S}_{\text{sys}}$ embeds $\mathcal{S}_{\text{res}}$.

***Where the multiplier is lost, and why it is model-dependent.*** Figure 4(b) separates the two ways $\phi$ falls: losses are either prefill-bound (SnapKV on Gemma 4 E4B and Qwen2.5 7B, DKV on Qwen2.5 7B) or peak-memory-bound (SnapKV on Gemma 4 E2B), and in no case are they predictable from a method's position in Figure 3. Figure 4(a) makes the model dependence explicit: the same integration inflates TTFT by 30.3×, 11.8× and 1.04× on the three models, and on the third its cost surfaces instead as a 2.47× peak-VRAM inflation ($\phi = 0.71$). Because the transform's cost rides on the model's prompt-processing profile, the same code path lands differently on different architectures. Methodologically this is the sharpest of our results: a single-model systems measurement of a compression mechanism does not transfer, and reporting one would have supported a confidently wrong generalisation in either direction.

### 5.4 Where the limit is the model, not the representation

Two features of the suite bound what its accuracy axis can resolve, and both are visible only because a dense run was paired with every query. First, every method scores 0.0% on RULER multi-step variable tracking on all three models — *including the uncompressed dense reference* (a single KIVI configuration on Qwen2.5 7B reaches 3.3%). Whatever fails there is not the representation, and without the paired dense run it would read as a compression failure. Because the task is one of five it depresses every reported average by roughly a fifth uniformly, changing no ranking. Second, dense accuracy on multi-needle retrieval is 32–33% on all three models, consistent with partial credit for locating a subset of the planted needles, so that task has little headroom to discriminate. Effectively three of five tasks carry the discriminative signal, and absolute accuracies here should be read with that in mind (Appendix C, Figure 6).

The same argument applies along context length. Over the full 2K–128K ladder on Gemma 4 E2B (Figure 7), dense accuracy is flat to 32K, falls at 64K and drops sharply at 128K, while the better compressed methods track it throughout without the gap widening systematically. The degradation beyond 64K is therefore shared with the uncompressed baseline and belongs to the model's usable context, not to compression — a distinction visible only because the dense reference was re-run at every context length rather than assumed flat.

## 6 Implications for Serving Systems

***Three numbers, not two.*** The minimum honest summary of a KV-compression mechanism is its effective bits per element *including* metadata, its task accuracy against a paired dense run of the same model, and its measured TTFT, throughput and peak memory relative to that same run. Each of the three carries information the other two do not: reporting a nominal bit-width understates real footprint by 6–23% in a family-dependent way; reporting a ratio without task measurement leaves an 18.7× accuracy range undetermined; reporting both without a systems measurement leaves a 30× prefill range undetermined.

***Which axis binds depends on the deployment.*** If the binding constraint is context length within a fixed memory budget, Axis 1 with honest metadata accounting is the right selector. If it is answer quality, Axis 2 must be measured on the target model, because the ordering of methods changed across our three. If it is interactive latency or serving cost, only Axis 3 is decisive, and it must be measured on the implementation that will actually be deployed — our SnapKV and DKV measurements illustrate, in both directions, how far an integration can sit from an algorithm's potential.

***Prefill cost deserves first-class accounting.*** A compression mechanism that shrinks the cache but adds a pass over the uncompressed prompt trades decode-side memory for prefill-side latency. In a disaggregated serving stack [25] that trade lands on a different machine from the one it benefits, and under chunked prefill [1] it competes directly with the decode batch for the same scheduling budget. Neither effect is expressible in a compression ratio. We report TTFT with device synchronisation around the full prefill-plus-transform-plus-first-token path, so that transform cost is

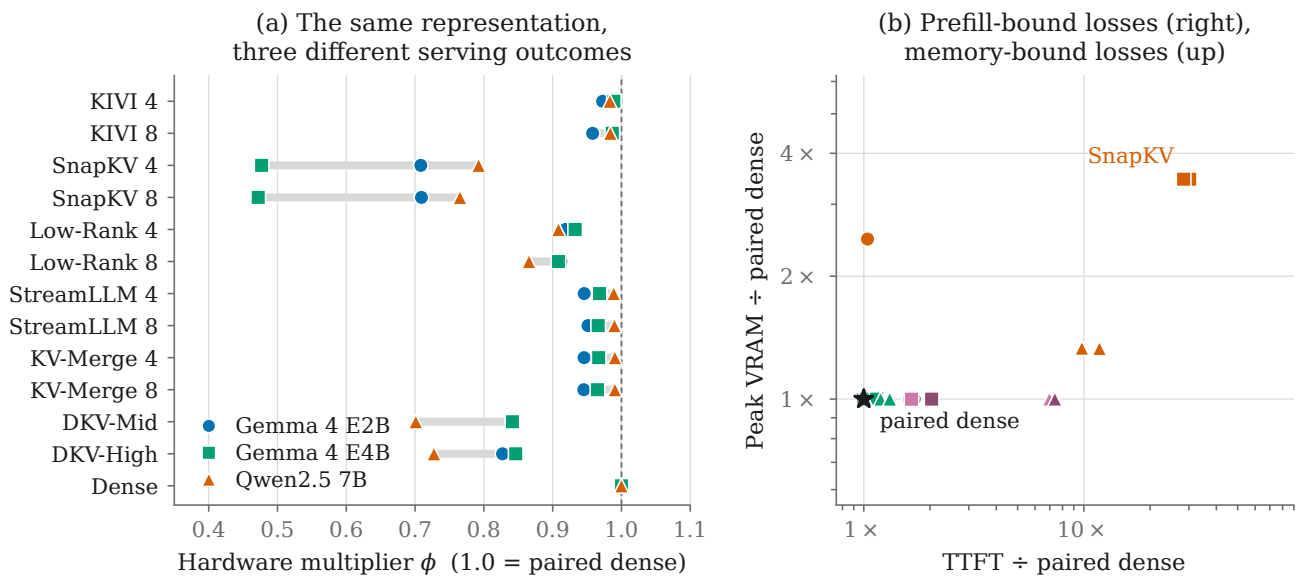



**Figure 4. The serving cost of a representation is a property of the deployment, not of the representation. (a)** $\phi$ for every operating point, one marker per model; a long grey bar is a method whose serving behaviour changed substantially when only the model changed. **(b)** The same points resolved into the two ways $\phi$ is lost: to the right are prefill-bound losses, upward are peak-memory-bound losses. SnapKV loses on different axes on different models. Neither panel is predictable from Figure 3.

charged to the method that incurs it, and we recommend the same convention.

***Keep the tracks apart.*** We report $S_{\text{res}}$ and $\phi$ separately and decline to average them, because their disagreement is the finding: a single score would have concealed that a configuration ranked second on representation utility ranks tenth on serving.

## 7 Threats to Validity and Scope

We state these as constraints on what our evidence supports, not as future work.

1. **Implementation, not algorithm.** Axis 3 measures code paths. Where an upstream fused kernel was unavailable on our platform — SnapKV's patched attention, DKV's paged runtime — our numbers characterise the integration that ran and are not upper bounds on those algorithms (Table 1). This is intrinsic to serving measurement rather than incidental to our setup: there is no implementation-independent serving number.
2. **Analytic versus realised savings.** For KIVI and Low-Rank KV the footprint is analytically accounted while values are retained in a wider dtype, so their Part 1 savings are exact for the scheme but no device memory was freed, and their favourable $\phi$ reflects a cheap transform rather than a production integer kernel. For SnapKV, StreamingLLM and KV-Merging the cache physically shrinks and both tracks are measured end to end.
3. **Three models, two families, 2B–7B.** Method orderings already differ across Gemma 4 E2B/E4B and Qwen2.5 7B, which is itself a reason not to extrapolate to 13B+ or 70B+ models, to different attention layouts, or to other tokenisers.
4. **Single-GPU, single-stream.** One RTX 4070 SUPER, batch size one, no tensor or pipeline parallelism, no continuous batching, no paged attention. Multi-tenant serving changes the memory–latency trade-off substantially and our Axis 3 magnitudes do not transfer to it unexamined; what we do claim is that a divergence this large at batch size one cannot be assumed to disappear in a multi-tenant stack.
5. **Retrieval-weighted synthetic tasks.** The suite probes retrieval, associative lookup and short multi-hop reasoning, not summarisation, open-ended generation, agentic dialogue or code, where the information a compressed cache must preserve is distributed differently. Effectively three of five tasks carry the discriminative signal (§5.4).
6. **Statistical resolution.** Rank correlations are computed over 12–13 operating points per model and 36 pooled. They are descriptive summaries of a small sample, and we quote $p$-values only to indicate that no relationship was

detectable, never to assert that none exists. The matched-budget and matched-pair comparisons are direct observations and do not depend on them.

7. **The weights are conventions.** $\alpha$ and the $\phi$ exponents are reported, varied (Appendix B) and decomposable, but not derived from anything. Readers with different priorities should reweight from the component columns.

## 8 Related Work

***LLM inference and serving.*** Serving systems treat the KV cache as the resource to be managed: PagedAttention allocates it in pages to eliminate fragmentation [15], continuous batching raises occupancy [22], disaggregation [25] and chunked prefill [1] reshape when prefill and decode compete, IO-aware kernels reduce attention's memory traffic [6], CacheGen compresses the cache for transfer [17], and ZipServ compresses losslessly with hardware-aware kernels [9]. These works measure latency, throughput and memory carefully; they generally do not measure task quality, and not per representation.

***KV-cache compression.*** KIVI [19], KVQuant [12] and LLM.int8() [7] quantise; SnapKV [16], StreamingLLM [21], $H_2O$ [24] and Scissorhands [18] evict; Ge et al. [10] and PyramidKV [5] merge or allocate adaptively; low-rank and factorised approaches project [8, 20]. Each reports on its own task subset, at its own bit accounting, on its own hardware, typically against a baseline reproduced by its own authors. The literature is therefore rich in individually credible results that cannot be placed on a common axis.

***Long-context evaluation.*** Needle-in-a-Haystack [14], RULER [13], LongBench [4], L-Eval [3] and ∞-Bench [23] measure what a model can do with a long context, now reaching the million-token range [11]. They do not track what that context costs, and they evaluate uncompressed models, so they cannot separate a representation's contribution from the base model's.

***Evaluating compression across quality and systems.*** Closest to this paper, concurrent work by Agrawal and Mayer [2] benchmarks KV-cache optimisations on both task quality and system performance for long-context serving, and reports that compression ratio alone is a poor predictor of end-to-end performance. That their conclusion and ours agree — from different models (Llama-3.1-8B and Mistral-7B versus Gemma 4 E2B/E4B and Qwen2.5 7B), different workloads (LongBench-style versus retrieval-weighted probes to 128K) and different metrics — is evidence that the effect is not an artefact of either setup. Our study is complementary in three respects, each a measurement-design choice rather than a larger sweep. *(i) Query-level pairing:* every ratio we report is formed against a dense execution of the same prompt in the same session, which is what makes the matched-pair observation of Figure 1(b) possible — two methods held equal on both reported numbers, then differing 30.3× — rather than a correlation across a scatter. *(ii) Metadata-charged accounting:* we charge each representation for the state that makes it decodable, and show the correction is 6–23% and re-ranks methods, where a payload-only ratio compares unlike quantities across families. *(iii) Provenance and realisation:* we state per method which code path produced its numbers and whether its saving was physically realised or analytically accounted, which licenses reading Axis 3 as a statement about a deployment rather than about an algorithm. We additionally report the model-dependence of the serving penalty (§5.3). Table 4 in Appendix C places these practices side by side. It is not that any one community measures badly; the three measurements are rarely taken on the same queries, so their disagreements stay invisible.

## 9 Conclusion

Compression ratio, task accuracy and serving efficiency are three separate properties of a KV-cache compression mechanism, and in our measurements none of them reliably determines the others. Methods matched to within 0.19 payload bits per element differ by up to 18.7× in task accuracy; methods matched on *both* footprint and accuracy differ by 30.3× in prefill latency and 3.46× in peak memory; and over 36 operating points the footprint of a representation carries no detectable information about the hardware efficiency it realises. Charging representations for the metadata that makes them decodable — which the conventional accounting omits — shifts reported bitrates by 6–23% and reorders methods.

For a serving system this means the two numbers a compression paper reports are not sufficient to choose a mechanism, and the missing third is not recoverable from them. TriAxis is the instrument these findings required: paired per-query execution against a dense reference, an effective-bits accounting that includes metadata, and two deliberately unmerged tracks. We release the records and the scripts that regenerate every number and figure, so the accounting choices we argue for can be disagreed with directly.

## Generative AI Disclosure

In accordance with the ACM Policy on Authorship, we disclose that generative AI tools (large language models) were used as assistive tools for stylistic editing and for drafting prose under the authors' direct supervision. No generative AI tool was used to design the study, produce or analyse measurements, generate data, or select citations; all experimental methodology, measurements, analysis, figures and scientific claims are the authors' own, and the authors take full responsibility for the entire content of this paper. No generative AI tool is listed as an author.

# References


[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. https://www.usenix.org/conference/osdi24/presentation/agrawal arXiv:2403.02310.

[2] Nikita Agrawal and Ruben Mayer. 2026. Benchmarking KV-Cache Optimizations across Task Quality and System Performance for Long-Context Serving. *arXiv preprint arXiv:2607.05399* (2026). https://arxiv.org/abs/2607.05399

[3] Chenxin An, Shansan Gong, Ming Zhong, Xingjian Zhao, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu. 2023. L-Eval: Instituting Standardized Evaluation for Long Context Language Models. *arXiv preprint arXiv:2307.11088* (2023). https://doi.org/10.18653/v1/2024.acl-long.776

[4] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. https://doi.org/10.18653/v1/2024.acl-long.172

[5] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2024. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *arXiv preprint arXiv:2406.02069* (2024). https://arxiv.org/abs/2406.02069

[6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2205.14135

[7] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2208.07339

[8] DKV Project. 2026. DKV: Anchor + Low-Rank Differential KV-Cache Compression for Scalable Long-Context Inference. Preprint.

[9] Ruibo Fan, Xiangrui Yu, Xinglin Pan, Zeyu Li, Weile Luo, Qiang Wang, Wei Wang, and Xiaowen Chu. 2026. ZipServ: Fast and Memory-Efficient LLM Inference with Hardware-Aware Lossless Compression. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM. https://doi.org/10.1145/3779212.3790250

[10] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2023. Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs. *arXiv preprint arXiv:2310.01801* (2023). https://arxiv.org/abs/2310.01801

[11] Gemini Team. 2024. Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context. *arXiv preprint arXiv:2403.05530* (2024). https://arxiv.org/abs/2403.05530

[12] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2401.18079

[13] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654* (2024). https://arxiv.org/abs/2404.06654

[14] Greg Kamradt. 2023. Needle in a Haystack — Pressure Testing LLMs. https://github.com/gkamradt/LLMTest_NeedleInAHaystack.

[15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. ACM, 611–626. https://doi.org/10.1145/3600006.3613165

[16] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2404.14469

[17] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. ACM. https://doi.org/10.1145/3651890.3672274

[18] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2305.17118

[19] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2402.02750

[20] Pratyusha Sharma, Jordan T. Ash, and Dipendra Misra. 2024. The Truth is in There: Improving Reasoning in Language Models with Layer-Selective Rank Reduction. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2312.13558

[21] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2309.17453

[22] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[23] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. 2024. ∞-Bench: Extending Long Context Evaluation Beyond 100K Tokens. *arXiv preprint arXiv:2402.13718* (2024). https://doi.org/10.18653/v1/2024.acl-long.814

[24] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2306.14048

[25] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 193–210. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

# A Scoring Properties

**Proposition A.1.** *Let $\mathcal{S}_{res}(Q,R) = \alpha Q + (1-\alpha)R$ with $\alpha \in (0,1)$. On $[0,100]^2$, $\mathcal{S}_{res}$ is strictly increasing in each argument, bounded in $[0,100]$, and Pareto consistent.*

*Proof.* *Monotonicity*: $\partial\mathcal{S}_{\text{res}}/\partial Q = \alpha > 0$ and $\partial\mathcal{S}_{\text{res}}/\partial R = 1-\alpha > 0$. *Boundedness*: $\mathcal{S}_{\text{res}}$ is a convex combination of $Q$ and $R$, so $\min(Q,R) \le \mathcal{S}_{\text{res}} \le \max(Q,R)$, giving $\mathcal{S}_{\text{res}} \in [0,100]$ with the extremes attained at $(0,0)$ and $(100,100)$. *Pareto consistency*: if $Q_A \ge Q_B$ and $R_A \ge R_B$ with at least one strict, then $\mathcal{S}_{\text{res}}(Q_A,R_A) - \mathcal{S}_{\text{res}}(Q_B,R_B) = \alpha(Q_A-Q_B) + (1-\alpha)(R_A-R_B) > 0$. □

# B Alternative Formulations and $\alpha$ Sensitivity

We considered multiplicative ($Q^\alpha R^{1-\alpha}$), harmonic and threshold-gated alternatives. All satisfy monotonicity and Pareto consistency; the multiplicative and harmonic forms additionally satisfy a substitution-resistance property that the linear form does not, in that a zero on either axis annihilates the score. We chose the linear form regardless, for two reasons. First, annihilation is undesirable here: the dense reference has $R_{\text{mem}} = 0$ by definition and would score zero under any multiplicative form, removing the one anchor every other row is measured against. Second, a linear score can be decomposed back into its inputs by inspection, which matters more for a measurement report intended to be argued with than a marginally better-behaved aggregate would. Figure 5 shows the ranking is stable over $\alpha \in [0.4, 0.9]$, so the choice of weight is not doing hidden work.

# C Per-Task, Per-Context and Per-Operating-Point Detail

**Table 4. The disagreement between axes is invisible unless the axes are measured on the same query.** "Anchor" means an uncompressed run of the *same* model is reported; "paired" means that run is on the *same* query. △ = sometimes, inconsistently. Discussed in §8.

| Practice | Qual. | Mem. | Lat. | Anch. | Pair |
|---|---|---|---|---|---|
| Long-context suites | ✓ | ✗ | ✗ | ✗ | ✗ |
| Synthetic probes (NIAH, RULER) | ✓ | ✗ | ✗ | ✗ | ✗ |
| Serving-system profilers | ✗ | ✓ | ✓ | ✗ | ✗ |
| Compression method papers | ✓ | △ | △ | △ | ✗ |
| Quality + systems benchmarks | ✓ | ✓ | ✓ | △ | ✗ |
| **This work** | ✓ | ✓ | ✓ | ✓ | ✓ |

Figure 6 resolves accuracy by task and Figure 7 by context length; both are discussed in §5.4. Figure 8 gives the Part 1 ordering at every operating point and Figure 9 the quality–memory frontier computed within each model. We compute the frontier per model rather than pooling: accuracy differences between a 2B and a 7B model reflect base-model capability, and a frontier drawn across them would be an artefact. The frontiers are short — on Gemma 4 E2B a single 4-bit configuration dominates everything else, matching dense accuracy at roughly a quarter of the bits — which is itself informative about how much headroom remains at these scales.

# D Measurement Protocol

***Execution.*** Prefill is executed in 4,096-token chunks. The context transform is applied to the resident PyTorch KV cache after prefill (or streaming during prefill, for the quantiser), and generation proceeds from the transformed cache. Decoding is greedy. TTFT is measured with device synchronisation around the full prefill-plus-transform-plus-first-token path, so transform cost is charged to the method that incurs it. Peak device memory is the maximum allocated over the whole query.

***Determinism.*** Because the dense reference is re-executed in each of the two budget sweeps, every query carries a free replication of its own baseline. On Gemma 4 E4B and Qwen2.5 7B all 150 re-executions produced byte-identical output text. On Gemma 4 E2B, 64/75 produced identical text and exactly one query changed score — a multi-needle prompt at 128K. Run-to-run variance is therefore negligible at the resolution of our claims, and is smallest on the two models where the matched-pair evidence is strongest.

***Aggregation.*** Task quality aggregates as the arithmetic mean of $Q_{\text{abs}}$ over queries. Memory aggregates by bytes rather than by query, $\bar{R}_{\text{mem}} = 100\,(1 - \sum_i M_{i,\text{method}} / \sum_i M_{i,\text{dense}})$, so that the long contexts where compression matters are not diluted by short ones. A query that failed to complete is charged the dense footprint and scores zero rather than being dropped, and is additionally visible in the separately reported coverage $C$.

***What is charged to metadata.*** Quantisation scales and zero points; SVD factor matrices; anchor and basis state for the factorised representations; 32-bit token position indices for eviction methods; and dtype-alignment padding where the scheme requires it. Partial groups that cannot be quantised at the target width are charged at 16 bits.

# E Artifact Appendix

***Scope.*** The artifact reproduces every table, statistic and figure in this paper from the released per-query records. It does *not* require re-executing the study: the 3,150 paired executions took approximately three GPU-days on the machine described in §4, and the raw records are shipped so that the analysis is reproducible in minutes on a CPU-only machine.

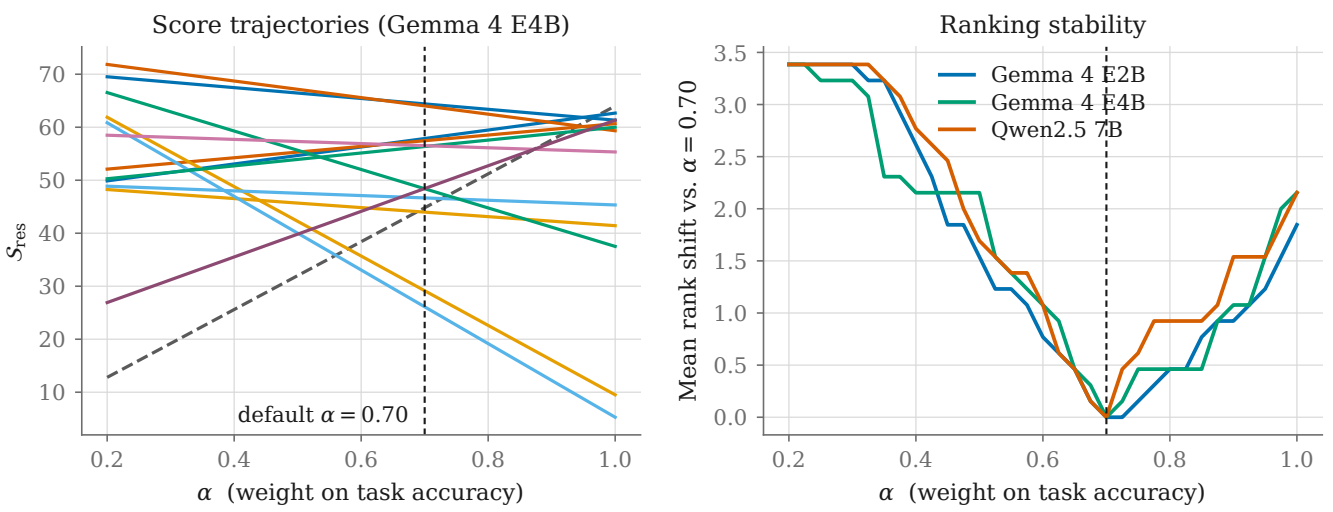



**Figure 5. The default weight is not load-bearing.** Score trajectories and mean rank displacement relative to $\alpha = 0.70$.

| | Single NIAH | Multi NIAH | RULER KV | Multi-hop QA | Var. track. |
|---|---|---|---|---|---|
| KIVI | 97 | 32 | 66 | 87 | 1 |
| SnapKV | 94 | 32 | 61 | 88 | 0 |
| Low-Rank | 51 | 17 | 43 | 70 | 0 |
| StreamLLM | 50 | 17 | 14 | 60 | 0 |
| KV-Merge | 37 | 13 | 10 | 38 | 0 |
| DKV-Mid | 86 | 27 | 51 | 91 | 0 |
| DKV-High | 100 | 32 | 60 | 87 | 0 |
| Dense | 100 | 33 | 64 | 87 | 0 |

Accuracy (%), mean of the three models

**Figure 6. One of our five tasks measures the base model, not the representation.** Accuracy by task, averaged over the three models and both budget levels. Variable tracking is at zero for every method *and* for the uncompressed dense reference, which identifies it as a base-model limitation rather than a compression failure; without a paired dense run it would read as a compression result.

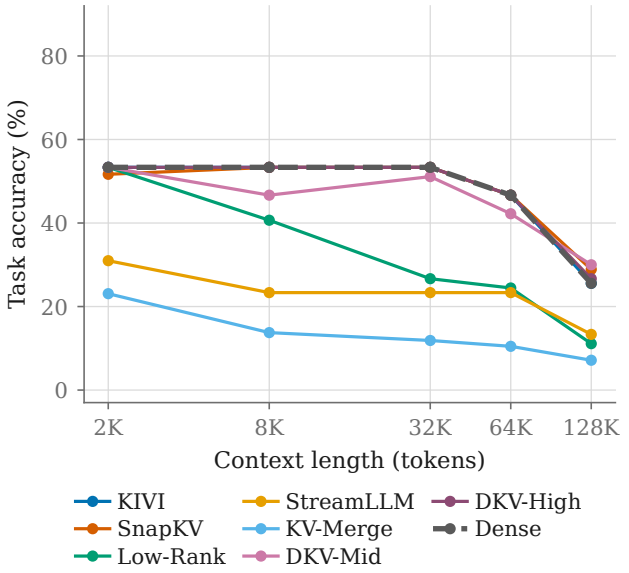



**Figure 7. The cliff beyond 64K belongs to the model, not to compression.** The 2K–128K ladder on Gemma 4 E2B. The dense reference (dashed) degrades together with the compressed methods, so the drop cannot be charged to the representation.

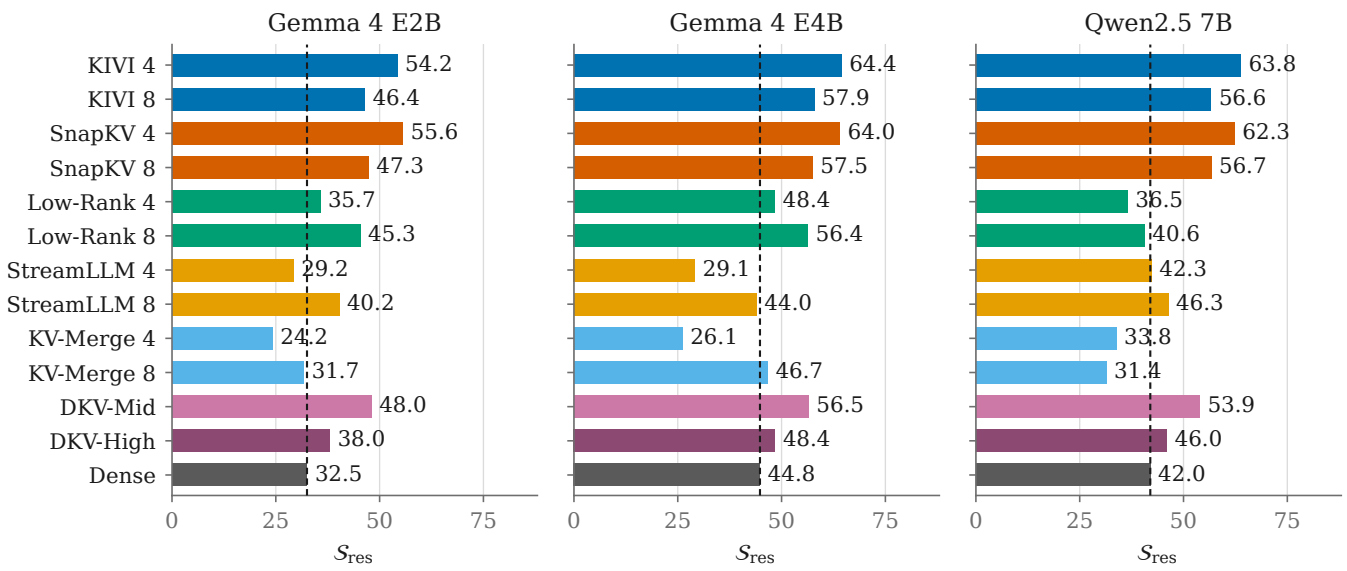



**Figure 8. Representation utility $S_{\text{res}}$ at every operating point,** with the dense reference marked (dashed). Dense is not last: at $\alpha = 0.70$, a method that gives up a few accuracy points for a large memory saving outscores it.

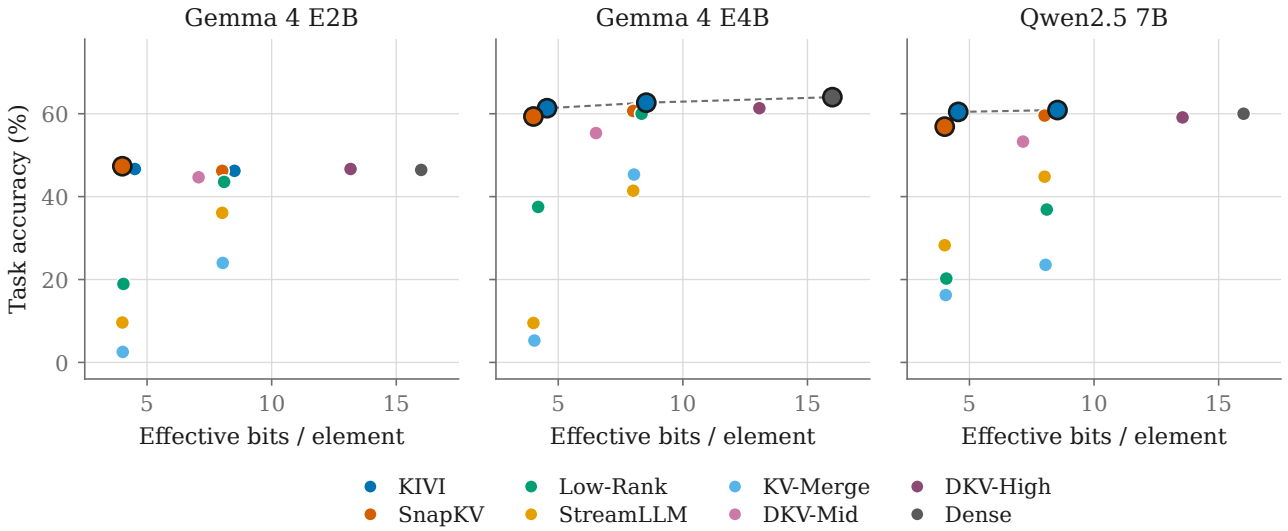



**Figure 9. Quality–memory frontier, computed within each model.** Effective bits include metadata. Outlined markers are non-dominated. Frontiers are not drawn across models, because accuracy differences between models reflect base-model capability rather than the representation.

***Contents.*** (i) The harness, including the adapter interface and the reference implementations noted as "harness" in Table 1; (ii) the per-query records for all three models, one JSON line per execution, carrying the task score, analytic footprint, TTFT, decode throughput and peak device memory for both the dense and the compressed run; (iii) four analysis scripts.

***Reproduction.*** The scripts regenerate, in order, every number in the tables (`build_tables.py`), every statistic quoted in §5 (`decoupling_analysis.py`) and every figure (`make_figures.py`), reading only the released records. A fourth script, `verify_claims.py`, re-derives each quantity quoted in the prose from those outputs and asserts that the manuscript source contains the matching value; it exits nonzero on any drift. No number in this paper is transcribed by hand.

***Re-executing the study.*** Re-running the measurements requires a CUDA GPU with at least 12 GB of device memory and access to the three open-weight checkpoints named in §4. Absolute latencies will differ on other hardware; the paired ratios are the quantities intended to be comparable.